MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name	Roll No	Class	Course	Module	Website
Anto Siju	14	CSA	PCCST501 – Computer Networks	1	youtube.com (YouTube)

1. Introduction

When a user types www.youtube.com and presses Enter, a rapid sequence of events unfolds across the layers of the Internet's protocol stack — name resolution, connection establishment, secure data exchange, and rendering — usually completing in under a second. This report traces that journey using YouTube as the running example.

2. The TCP/IP Protocol Stack

The TCP/IP model organizes communication into four layers, each handing data to the layer below (sending) or above (receiving):

Layer	Protocols	Role
Application	HTTP/HTTPS, DNS, FTP, SMTP	Formats and requests the actual web page data (browser + DNS operate here)
Transport	TCP, UDP	Ensures reliable, ordered, error-checked delivery; manages the three-way handshake
Internet	IP, ICMP	Adds source/destination IP addresses; routers use this to forward packets
Network Access	Ethernet, Wi-Fi (802.11), ARP	Converts packets to physical signals (frames/bits) for the local hop

Figure 1: TCP/IP Four-Layer Protocol Stack

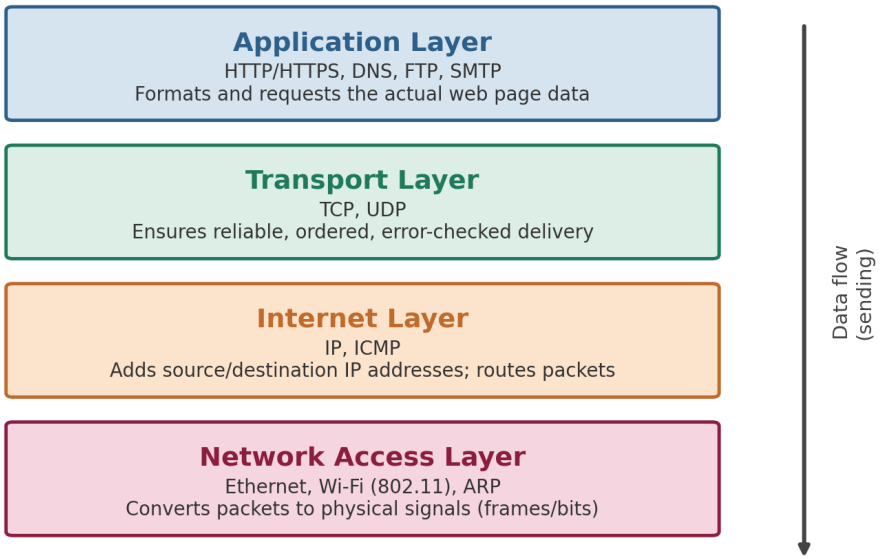

Figure 1: TCP/IP four-layer protocol stack

Encapsulation: as the request travels down the stack, each layer wraps the data from above with its own header — the Application layer's HTTP request is wrapped in a TCP segment (ports), then an IP packet (IP addresses), then a frame (MAC addresses) — before conversion to signals. At YouTube's server, de-encapsulation reverses this, stripping each header to recover the original request.

TCP vs UDP: Loading the YouTube page uses TCP (via HTTPS) rather than UDP because a corrupted or incomplete page is unacceptable — the user needs complete, correctly-ordered HTML. UDP is reserved for cases where speed matters more than perfection, such as live video calls or DNS lookups themselves.

3. Role of DNS

Browsers and routers route traffic using numeric IP addresses, not domain names. DNS is the Internet's directory service that performs this translation:

- The browser checks its own cache, then the OS cache, for a recent record of youtube.com.
- If not cached, a DNS resolver (ISP or public, e.g. 8.8.8.8) is queried.
- The resolver walks the hierarchy: a root server points to the .com TLD server, which points to YouTube's authoritative name server.
- The authoritative server returns the IP address (e.g. 142.250.193.174).
- The IP is cached locally (subject to a TTL) and handed to the browser to proceed.

Figure 2: Hierarchical DNS Resolution for youtube.com

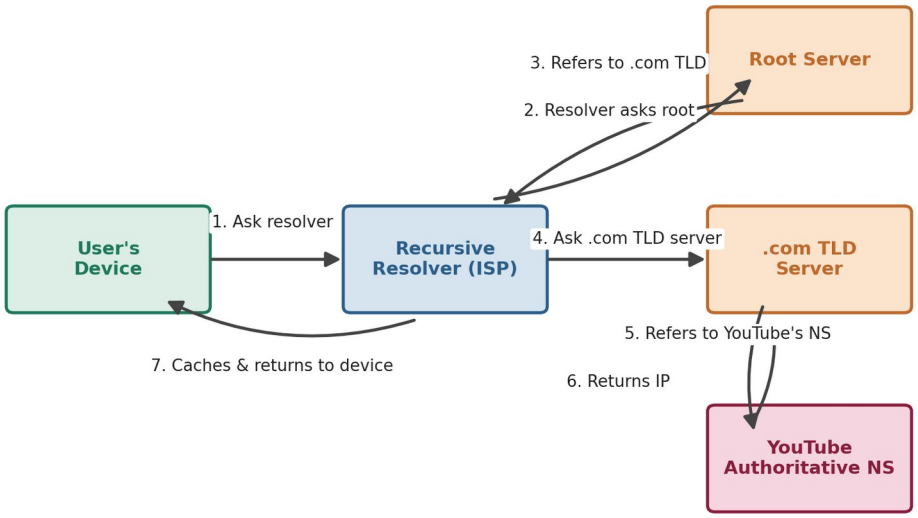

Figure 2: Hierarchical DNS resolution process for youtube.com

Without DNS, users would need to memorize numeric addresses for every site — hence its nickname, the "phonebook of the Internet."

Record	Purpose
A	Maps a domain name to an IPv4 address (e.g. youtube.com → 142.250.193.174)
AAAA	Maps a domain name to an IPv6 address
CNAME	Aliases one domain name to another canonical domain name
NS	Specifies which name servers are authoritative for the domain
TTL	Not a record type, but a value telling resolvers how long to cache a record

4. HTTP: Requesting and Receiving the Webpage

Once the browser has YouTube's IP and a TCP connection is open, HTTP(S) fetches the page: (1) the browser sends a GET request, e.g. GET /watch?v=example HTTP/1.1, with headers such as Host and User-Agent; (2) the server returns a status line (e.g. 200 OK), headers, and the HTML body; (3) the browser parses the HTML and issues further GET requests for CSS, JS, images, and fonts; (4) once enough resources arrive, the page renders.

HTTPS wraps this same request/response cycle inside a TLS encryption layer. Before any HTTP data is exchanged, browser and server perform a TLS handshake to agree on encryption keys and verify the server's certificate — preventing attackers from reading or tampering with the request or video content.

Code	Meaning	Example on YouTube
200	OK – request succeeded	Video page loads normally

Code	Meaning	Example on YouTube
301/302	Redirect to another URL	youtube.com → www.youtube.com
304	Not modified – use cache	Re-loading a cached thumbnail
404	Resource not found	A broken or mistyped video link
500	Internal server error	A temporary fault on YouTube's servers

5. How TCP Provides Reliable Delivery

- Three-way handshake – SYN, SYN-ACK, ACK establish a connection before data is exchanged.
- Sequence numbers – every byte is numbered so segments can be reassembled in order.
- Acknowledgements (ACKs) – the receiver confirms receipt; missing ACKs signal loss.
- Retransmission – TCP resends a segment if its ACK times out.
- Flow control – a sliding window prevents overwhelming a slower receiver.
- Checksums – detect corrupted data so it can be discarded and retransmitted.

6. Client–Server or Peer-to-Peer?

YouTube follows a Client–Server architecture: the browser (client) always initiates the request, and YouTube's centralized data-center servers respond, store content, and serve pages/streams. Clients never fetch video content directly from one another. This contrasts with Peer-to-Peer (P2P) systems like BitTorrent, where every node is both client and server and content is exchanged directly between peers.

Aspect	Client–Server (YouTube)	Peer-to-Peer (e.g. BitTorrent)
Roles	Fixed – client requests, server responds	Interchangeable – every node is both
Control	Centralized at data centers	Decentralized across peers
Scalability	Add more/larger servers	Grows automatically as peers join
Single point of failure	Yes, if server/data center is down	No, survives individual peer loss

7. HTTP vs FTP

Aspect	HTTP	FTP
Purpose	Transfers web pages/resources for browsing	Transfers files for storage/download
Connections	Single connection for requests and data	Separate control (port 21) and data (port 20)
State	Stateless by default (cookies add state)	Stateful – session persists
Authentication	Often none for public content	Typically requires username/password
Typical use	Loading a webpage like youtube.com	Uploading a site's files to a hosting server

FTP can run in active mode (server opens a connection back to the client) or passive mode (client initiates both connections — more firewall-friendly and the default in most modern clients). HTTP's single, simpler connection model is one reason it became the protocol of choice for everyday browsing, while FTP remains common for bulk file transfer and server administration.

8. How Cookies Improve User Experience

HTTP is stateless — the server does not naturally remember a user between requests. Cookies let the server store a small piece of data on the browser, sent back with every subsequent request to that domain.

Real-world example: signing in to a Google account and opening YouTube sets a session cookie. It keeps the user logged in while navigating from Home to a Watch page to Subscriptions without re-entering credentials, and lets YouTube remember preferences such as language, autoplay, and watch history.

9. End-to-End Communication Flow

Figure 3: End-to-End Communication Flow

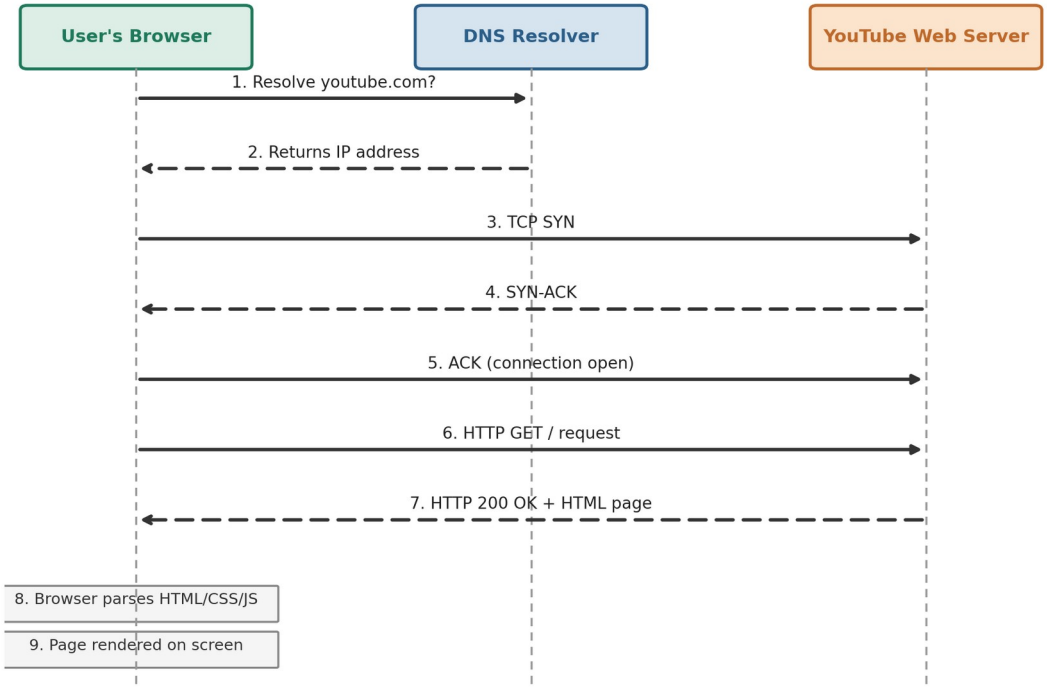

Figure 3: Complete browser-to-server communication flow for loading youtube.com

10. Conclusion

Loading a single webpage like youtube.com depends on a tightly coordinated stack of protocols working invisibly in the background. DNS turns a memorable name into a routable address, TCP guarantees data survives the journey intact, IP finds the path across networks, and HTTP structures the conversation between browser and server. Cookies layer statefulness on top of an inherently stateless protocol, enabling personalization and persistent logins. Every video view, login, and page load depends on these protocols functioning correctly, quickly, and securely — without this layered design, the modern web could not scale or remain reliable.